

OpenAI Agents SDK: From Zero to Your First Tool-Using Agent

A hands-on guide for beginners

This guide takes you from nothing to a working AI agent, in the order you should actually learn it:

1. **Setup** — get a project ready.
2. **OpenAI key integration** — the simplest possible agent (SDK's native, default provider).
3. **Gemini key integration** — swap in a different LLM provider.
4. **Runner and asyncio** — how an agent actually executes, and why it's async under the hood.
5. **Model configuration** — the three levels (Agent / Run / Global) for choosing which LLM to use.
6. **Basic tool use** — give your agent the ability to call Python functions with `@function_tool`.
7. **Model settings** — turn the model's knobs: temperature, tool choice, answer length.
8. **Local context** — hand your tools data about the user that the LLM never sees.
9. **Dynamic instructions** — build the system prompt at runtime instead of hardcoding it.
10. **Agent cloning** — spin specialised variants off one base agent.
11. **Tracing** — watch every step of a run on the traces dashboard.
12. **Agents as tools** — let one agent call another as a specialist.
13. **Handoffs** — transfer the whole conversation to a specialist agent.
14. **Advanced tool control** — stop the loop, gate tools, survive failures.
15. **Structured output** — get a typed object back instead of prose.
16. **Guardrails** — refuse bad input and bad output before they cost you.
17. **Lifecycle hooks** — get a callback at every stage of one agent's work.
18. **Run lifecycle hooks** — one monitor across every agent in a run.
19. **Custom runners** — put your own code around every run in the process.
20. **Chainlit** — give the whole thing a chat UI.
21. **Advanced handoffs** — rename, gate, log and trim a transfer.

A **Bonus** section at the end takes `run`, `run_sync` and `run_streamed` apart in detail.

Everything here is copy-paste ready. Type the code yourself rather than pasting — it sticks better.

Part 0 — Setup

You need **Python 3.13+** and **uv** (a fast Python package manager). If you don't have **uv**:

```
pip install uv
```

Create a project:

```
uv init hello-agent
cd hello-agent
uv add openai-agents python-dotenv
```

Create a `.env` file in the project root — this is where your API keys live. **Never commit this file.**

```
OPENAI_API_KEY=
GEMINI_API_KEY=
```

You'll fill these in as you go through Parts 1 and 2.

Part 1 — OpenAI Key Integration (the default provider)

The Agents SDK is built by OpenAI, so **OpenAI is the default provider** — no custom client setup required. You just need an API key.

Get a key

1. Go to <https://platform.openai.com/api-keys> and create a key.
2. Paste it into `.env`:

```
OPENAI_API_KEY=sk-...
```

Write the agent

Create `main.py`:

```
from agents import Agent, Runner
from dotenv import load_dotenv, find_dotenv

# Loads .env so OPENAI_API_KEY is available as an environment variable
load_dotenv(find_dotenv())

# The SDK finds OPENAI_API_KEY automatically — no client wiring needed
agent = Agent(
    name="Assistant",
    instructions="You are a helpful assistant.",
)

result = Runner.run_sync(agent, "Write a haiku about recursion in programming.")
print(result.final_output)
```

Run it:

```
uv run main.py
```

That's the whole integration. Three things happened:

- `Agent(...)` — defined *who* the agent is (a name + a system prompt via `instructions`).
- `Runner.run_sync(...)` — actually sent the prompt to the LLM and waited for the result.
- `result.final_output` — the agent's final text response.

Note: If you don't pass a `model=`, the Agent uses the SDK's built-in default OpenAI model. You'll learn to override this explicitly in Part 4.

Part 2 — Gemini Key Integration (a different provider)

The Agents SDK only *ships* with OpenAI wiring, but it can talk to **any provider that exposes an OpenAI-compatible Chat Completions API** — and Google's Gemini API does exactly that.

Get a key

1. Go to <https://ai.google.dev/gemini-api/docs/api-key> and create a Gemini API key.
2. Paste it into `.env`:

```
GEMINI_API_KEY=...
```

Write the agent

```
from agents import Agent, Runner, OpenAIChatCompletionsModel, AsyncOpenAI
import os
from dotenv import load_dotenv, find_dotenv

load_dotenv(find_dotenv())

# 1. Point an OpenAI-compatible client at Gemini's endpoint instead of OpenAI's
external_client = AsyncOpenAI(
    api_key=os.getenv("GEMINI_API_KEY"),
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

# 2. Wrap it as a Chat Completions model the SDK understands
llm_model = OpenAIChatCompletionsModel(
    model="gemini-2.5-flash",
    openai_client=external_client,
)

# 3. Hand that model to the Agent instead of using the default
agent = Agent(name="Assistant", model=llm_model)

result = Runner.run_sync(agent, "Welcome and motivate me to learn Agentic AI.")
print("AGENT RESPONSE:", result.final_output)
```

Why this works

- `AsyncOpenAI` is just an HTTP client shaped for OpenAI's API contract. Gemini implements that same contract at a different `base_url`, so the same client class works — you're not using any Google-specific SDK.
- `OpenAIChatCompletionsModel` tells the Agent "call this client using the Chat Completions format, with this specific model name."
- Everything else — `Agent`, `Runner`, `.final_output` — is identical to Part 1. **This is the whole point of the SDK's design:** the provider is just a pluggable `model`, the rest of your agent code never changes.

Part 3 — Understanding `Runner` and `asyncio`

You've used `Runner.run_sync(...)` twice without asking what it does. Time to open that up.

Why agents are async in the first place

Every call an agent makes to an LLM is a **network request** — it goes out over HTTP and the program sits waiting for a response. That's *I/O-bound* waiting, not CPU work. Python's `asyncio` exists so a program can do other useful things (or run several LLM calls concurrently) instead of blocking dead while it waits on the network. The Agents SDK is written natively with `async/await` for this reason.

The three ways to run an agent

Method	Signature	When to use it
<code>Runner.run(...)</code>	<code>async def run(...)</code> — must be <code>awaited</code>	Inside an <code>async def</code> function — real applications, servers, concurrent agents
<code>Runner.run_sync(...)</code>	plain function	Scripts, notebooks, quick tests — internally starts an event loop and blocks for you
<code>Runner.run_streamed(...)</code>	returns a streaming result	When you want to show tokens/events as they arrive, instead of waiting for the whole answer

The async version, properly

```
import asyncio
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="You only respond in haikus.")

async def main():
    result = await Runner.run(agent, "Tell me about recursion in programming.")
    print(result.final_output)

if __name__ == "__main__":
    asyncio.run(main())
```

Breaking this down:

- `async def main()` — declares a coroutine, a function that can pause and resume instead of blocking.
- `await Runner.run(...)` — pause `main()` here until the LLM responds, without freezing the whole program.
- `asyncio.run(main())` — creates the event loop, runs `main()` to completion, and shuts the loop down. This is the standard entry point for any async Python script.

`Runner.run_sync(agent, "...")` you used in Parts 1–2 is a convenience wrapper: it drives that same `Runner.run(...)` coroutine on an event loop and blocks until the result is ready. (It does not call `asyncio.run()` — the Bonus section at the end shows what it really does.) **Use `run_sync` for quick scripts. Use `run` inside real async apps** — e.g. a FastAPI endpoint, or when you want to fire off multiple agents at once with `asyncio.gather(...)`.

Try it: run two agents concurrently

```
import asyncio
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="Answer in one short sentence.")

async def main():
    results = await asyncio.gather(
        Runner.run(agent, "What is Python?"),
        Runner.run(agent, "What is an AI agent?"),
    )
    for r in results:
        print(r.final_output)

asyncio.run(main())
```

Both prompts are in flight to the LLM provider at the same time — this is only possible because `Runner.run` is async.

Part 4 — Model Configuration: Agent, Run, and Global Levels

You already did **Agent-level** configuration in Part 2 (passing `model=` straight into `Agent(...)`). That's the SDK's default provider is OpenAI; you can override it at three different scopes.

We will always use Agent-level configuration in this course — it lets each agent in a multi-agent system use whichever model fits its job (e.g., a cheap fast model for routing, a stronger model for reasoning).

1. Agent Level — override per agent

```
from agents import Agent, OpenAIChatCompletionsModel, AsyncOpenAI, Runner, set_tracing_disable

client = AsyncOpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)
set_tracing_disabled(disabled=True) # OpenAI's tracing dashboard expects an OpenAI key; disab

agent = Agent(
    name="Assistant",
    instructions="You only respond in haikus.",
    model=OpenAIChatCompletionsModel(model="gemini-2.0-flash", openai_client=client),
)
```

Different agents in the same app can each have their own `model=`.

2. Run Level — override for one call

```
from agents.run import RunConfig

config = RunConfig(
    model=OpenAIChatCompletionsModel(model="gemini-2.0-flash", openai_client=external_client),
    model_provider=external_client,
    tracing_disabled=True,
)

agent = Agent(name="Assistant", instructions="You are a helpful assistant")
result = Runner.run_sync(agent, "Hello, how are you.", run_config=config)
```

The agent itself is unconfigured; the override applies only to this specific `Runner.run_sync(...)` call — useful for A/B testing a model without touching agent definitions.

3. Global Level — override for the whole process

```
from agents import set_default_openai_client, set_default_openai_api, set_tracing_disabled

set_tracing_disabled(True)
set_default_openai_api("chat_completions")
set_default_openai_client(external_client)

agent = Agent(name="Assistant", instructions="You are a helpful assistant", model="gemini-2.0-")
```

Every agent created afterward uses this client by default, unless it — or its run — overrides it locally. Agent-level and Run-level configuration both win over Global.

Precedence, most specific wins: Agent level > Run level > Global level.

Part 5 — Basic Tool Use with `@function_tool`

So far your agent can only talk. A **tool** lets it *do* something — call a Python function, hit an API, query a database — and use the result to answer better.

The core idea

You write a normal Python function. The `@function_tool` decorator turns it into something the LLM can call:

- The **function name** becomes the tool's name.
- The **docstring** becomes the tool's description (this is what tells the LLM *when* to use it).
- The **type hints** are used to auto-generate the JSON schema for the tool's parameters.

```

from agents import Agent, Runner, function_tool

@function_tool
def get_weather(city: str) -> str:
    """Get the current weather for a given city."""
    return f"The weather in {city} is sunny and 25°C."

agent = Agent(
    name="Weather Assistant",
    instructions="You are a helpful assistant. Use tools when they help you answer.",
    tools=[get_weather],
)

result = Runner.run_sync(agent, "What's the weather like in Karachi?")
print(result.final_output)

```

What actually happens when you run this

1. `Runner.run_sync` sends your prompt **and** the tool's schema (name, description, parameters) to the LLM.
2. The LLM decides the question needs live data it doesn't have, and responds with "call `get_weather` with `city="Karachi"`" instead of a text answer.
3. The SDK intercepts that, **runs your actual Python function** with those arguments, and sends the return value back to the LLM as a new message.
4. The LLM reads the tool's result and writes the final natural-language answer.
5. `Runner.run_sync` only returns to you once this whole loop is finished — `result.final_output` is the final answer, tool calls included.

You can register multiple tools; the agent decides on its own which (if any) to call, and can even chain several tool calls in one turn.

Try it: add a second tool

```

@function_tool
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b

agent = Agent(
    name="Assistant",
    instructions="Use tools when they help you answer.",
    tools=[get_weather, add],
)

result = Runner.run_sync(agent, "What's 42 plus 58, and what's the weather in Lahore?")
print(result.final_output)

```

Watch the model correctly call both tools in the same turn and combine both results into one answer.

Beyond `@function_tool`

`@function_tool` is for **your own** Python functions. The SDK also ships **hosted/built-in OpenAI tools** — like `WebSearchTool` and `FileSearchTool` — that run on OpenAI's servers instead of your machine. Those are out of scope for this guide; `@function_tool` is the one you'll use 90% of the time, and it's the foundation everything else builds on.

Part 6 — Model Settings: Temperature, Tool Choice, and Length

Part 4 decided **which** model runs your agent. `ModelSettings` decides **how** that model behaves once it's running — how creative it is, whether it's allowed to reach for a tool, and how long its answer can get. Same agent, same prompt, different output.

The three settings to learn first

- **temperature** — the creativity dial. Low keeps answers focused and repeatable; high makes them varied and surprising.
- **tool_choice** — whether the model may, must, or must not call a tool.
- **max_tokens** — a hard ceiling on how long the answer can be.

You pass them as one object to the Agent:

```
from agents import Agent, Runner, ModelSettings

question = "Tell me about AI in 2 sentences."

for temperature in (0.1, 1.9):
    agent = Agent(
        name="Assistant",
        instructions="You are a helpful assistant.",
        model=llm_model,
        model_settings=ModelSettings(temperature=temperature),
    )
    print(f"\ntemperature={temperature}:")
    print(Runner.run_sync(agent, question).final_output)
```

Run it twice. The **0.1** answer will barely change between runs; the **1.9** answer will come out different every time. Use low temperature for maths, facts and instruction-following; high for brainstorming and story writing.

Note: The usable range depends on the provider. Gemini accepts a **temperature** up to **2.0**, so **1.9** is a legal (and very loose) setting here.

Forcing — or forbidding — tool use

tool_choice overrides the model's own judgement about the tools you registered in Part 5.

```

from agents import function_tool

@function_tool
def calculate_area(length: float, width: float) -> str:
    """Calculate the area of a rectangle."""
    return f"Area = {length} × {width} = {length * width} square units"

question = "What's the area of a 5x3 rectangle?"

for choice in ("auto", "required", "none"):
    agent = Agent(
        name=f"{choice} agent",
        instructions="You are a helpful assistant.",
        model=llm_model,
        tools=[calculate_area],
        model_settings=ModelSettings(tool_choice=choice),
    )
    print(f"\ntool_choice={choice}:")
    print(Runner.run_sync(agent, question).final_output)

```

- **"auto"** — the model decides. This is the default, and what Part 5 was already doing.
- **"required"** — the model **must** call a tool before it may answer. Useful when guessing is worse than failing (calculations, lookups).
- **"none"** — the model still sees the tool's schema but is forbidden from calling it. It will answer from its own head, and you'll see it do the arithmetic itself.

Try it: cap the answer length

```

brief_agent = Agent(
    name="Brief Assistant",
    instructions="You are a helpful assistant.",
    model=llm_model,
    model_settings=ModelSettings(temperature=0.2, max_tokens=100),
)

```

max_tokens cuts the response off at the limit — it does **not** ask the model to be concise. If the answer needs more room than you gave it, it stops mid-sentence. Say "answer in two sentences" in the **instructions** when you want brevity; use **max_tokens** when you want a cost ceiling.

The rest of the knobs

ModelSettings also carries **parallel_tool_calls** (let the agent fire several tools at once instead of one at a time), **top_p**, **frequency_penalty** and **presence_penalty**. Leave all of them at their defaults until you have a specific reason not to — changing one setting at a time is the only way to tell what actually helped.

Part 7 — Local Context: Data Your Tools See and the LLM Doesn't

Your tools so far have been self-contained: everything they needed arrived as an argument from the LLM. Real tools need things the LLM shouldn't be inventing — who the logged-in user is, their ID, a database handle, a logger. **Local context** is how you pass those in.

The core idea

You define a plain Python object (a dataclass is the usual choice), hand it to `Runner.run(..., context=...)`, and the SDK wraps it in a `RunContextWrapper` that every tool in that run receives as its first parameter.

```
import asyncio
from dataclasses import dataclass
from agents import Agent, Runner, RunContextWrapper, function_tool

@dataclass
class UserInfo:
    name: str
    uid: int

@function_tool
async def fetch_user_age(wrapper: RunContextWrapper[UserInfo]) -> str:
    """Returns the age of the user."""
    return f"User {wrapper.context.name} is 47 years old"

async def main():
    user_info = UserInfo(name="Ali", uid=123)

    agent = Agent[UserInfo](
        name="Assistant",
        model=llm_model,
        tools=[fetch_user_age],
    )

    result = await Runner.run(
        starting_agent=agent,
        input="What is the age of the user?",
        context=user_info,
    )
    print(result.final_output)

asyncio.run(main())
```

Note what the prompt does **not** say: it never mentions Ali. The LLM asks for the user's age, the SDK runs your function, and your function reads the name out of `wrapper.context`.

The part that surprises people

The context is **never sent to the LLM**. Look at the schema the SDK generated for that tool:

```
print(fetch_user_age.params_json_schema)
# {'properties': {}, 'title': 'fetch_user_age_args', 'type': 'object', ...}
```

Empty. The **wrapper** parameter is stripped out before the tool description goes to the model, so the model calls **fetch_user_age()** with no arguments and can't see — or invent — the user's identity. Compare that with **calculate_area** in Part 6, where **length** and **width** are in the schema because the model is meant to supply them.

Two rules follow from this:

- Every tool, hook and callback in a single run must use the **same** context type — that's why the agent is written **Agent[UserInfo]**.
- If you want the LLM to *know* something, it belongs in **instructions** or in the input message, not in the context.

Try it: a second tool on the same context

```
@dataclass
class UserInfo:
    name: str
    uid: int
    location: str = "Pakistan"

@function_tool
async def fetch_user_location(wrapper: RunContextWrapper[UserInfo]) -> str:
    """Returns the location of the user."""
    return f"User {wrapper.context.name} is from {wrapper.context.location}"
```

Register both tools, then ask **"What is the age of the user, and where are they from?"** — one context object, two tools reading different fields out of it, no user data anywhere in the prompt.

Part 8 — Dynamic Instructions: A System Prompt That Changes

Every agent so far had its **instructions** fixed at definition time. The SDK also accepts a **function** there. It runs at the start of each turn and returns the string to use as that turn's system prompt — so the agent's persona can depend on who is asking, what time it is, or how far the conversation has gone.

The core idea

The function takes exactly two parameters — the context wrapper from Part 7 and the agent itself — and returns a **str**.

```

from dataclasses import dataclass
from agents import Agent, Runner, RunContextWrapper

@dataclass
class UserInfo:
    name: str
    uid: int

def special_prompt(ctx: RunContextWrapper[UserInfo], agent: Agent) -> str:
    return (
        f"You are a math expert. User: {ctx.context.name}, Agent: {agent.name}. "
        "Please assist with math-related queries."
    )

math_agent = Agent[UserInfo](
    name="Genius",
    instructions=special_prompt,
    model=llm_model,
)

user = UserInfo(name="Ali", uid=123)
result = Runner.run_sync(math_agent, "What is 17 × 23?", context=user)
print(result.final_output)

```

- `ctx.context` is the object you passed as `context=` in Part 7 — this is where local context and instructions meet.
- `agent` is the agent being run, so `agent.name` and `len(agent.tools)` are available while you build the prompt.
- The function may also be `async def`; the SDK awaits it, so you can hit a database before returning the string.

Note: The callable must accept **exactly two** parameters. Give it one or three and the SDK raises `TypeError: 'instructions' callable must accept exactly 2 arguments (context, agent)`.

Try it: instructions that remember

Any callable object works, not just a function — which is how you keep state between runs:

```

class StatefulInstructions:
    def __init__(self):
        self.count = 0

    def __call__(self, ctx: RunContextWrapper, agent: Agent) -> str:
        self.count += 1
        if self.count == 1:
            return "You are a learning assistant. First interaction – be welcoming."
        return f"You are a learning assistant. Interaction #{self.count} – be brief."

agent = Agent(name="Stateful", instructions=StatefulInstructions(), model=llm_model)

for _ in range(3):
    print(Runner.run_sync(agent, "Tell me about AI in one line").final_output)

```

The first answer greets you; the next two get progressively terser, because the object counted the runs.

What the context does not hold

`RunContextWrapper` carries `.context` (your own object), `.usage` (token counts so far) and `.turn_input`. It does **not** carry a list of past messages. Code like `len(getattr(ctx, "messages", []))` compiles, runs, and quietly returns `0` forever — the branch you wrote for a long conversation never fires. If you want to react to conversation length, count it yourself in your own context object, the way `StatefulInstructions` above does.

Part 9 — Cloning Agents: One Base, Many Variants

Parts 6 and 8 handed you knobs: temperature, tool choice, instructions. Once you have an agent you like, you rarely want to rewrite it from scratch just to turn one of those. `agent.clone(...)` copies an agent and overrides only the fields you name.

The core idea

```
from agents import Agent, Runner, ModelSettings

base_agent = Agent(
    name="BaseAssistant",
    instructions="You are a helpful assistant.",
    model=llm_model,
    model_settings=ModelSettings(temperature=0.7),
)

creative_agent = base_agent.clone(
    name="CreativeAssistant",
    instructions="You are a creative writing assistant. Use vivid language.",
    model_settings=ModelSettings(temperature=0.9),
)

precise_agent = base_agent.clone(
    name="PreciseAssistant",
    instructions="You are a precise, factual assistant.",
    model_settings=ModelSettings(temperature=0.1),
)

for agent in (base_agent, creative_agent, precise_agent):
    print(f"\n{agent.name}:")
    print(Runner.run_sync(agent, "Describe a sunset.").final_output)
```

Anything you don't pass is carried over from the base — neither clone repeats `model=llm_model`, and both still run on Gemini. This is how you build a routing model, a reasoning model and a summariser out of one definition.

The shallow-copy trap

`clone()` is `dataclasses.replace` underneath, which makes a **shallow** copy. A list you don't override isn't copied — the base and the clone hold the *same* list object.

```
base = Agent(name="Base", instructions="Be helpful.", tools=[get_weather])
shared_clone = base.clone(name="SharedClone")

base.tools.append(add)          # appended through the original...
print(len(shared_clone.tools))  # 2 — the clone grew a tool too

independent = base.clone(name="Independent", tools=[*base.tools])
base.tools.append(calculate_area)
print(len(independent.tools))   # 2 — its own list, unaffected
```

The rule: **pass a fresh list whenever you want independence** — `tools=[*base.tools, extra_tool]`. The same applies to `handoffs`, `mcp_servers` and the guardrail lists.

Try it: settings are replaced, not merged

```
from dataclasses import replace

base = Agent(
    name="Base",
    instructions="Be helpful.",
    model_settings=ModelSettings(temperature=0.7, max_tokens=500),
)

careless = base.clone(model_settings=ModelSettings(temperature=0.1))
print(careless.model_settings.max_tokens)    # None – the 500 was dropped

careful = base.clone(model_settings=replace(base.model_settings, temperature=0.1))
print(careful.model_settings.max_tokens)    # 500 – every other field kept
```

A `ModelSettings` handed to `clone()` swaps the whole object, so any field you leave out silently falls back to its default. When you mean to change one setting and keep the rest, `dataclasses.replace` on the base's settings is the safe move.

Part 10 — Tracing: Seeing What Your Agent Actually Did

Since Part 4 you've been writing `set_tracing_disabled(disabled=True)` without being told what you were switching off. This is it. Tracing is on by default, and it records every step of a run so you can open it up afterwards instead of guessing from the final answer.

Trace and span

A **trace** is one complete workflow — everything that happened between your prompt and the final output. A **span** is one step inside it: an LLM call, a tool call, one agent's turn. A run that calls `get_weather` and then answers produces one trace containing several spans, and the dashboard draws them as a timeline.

Turning it on while using Gemini

Here's the wrinkle that trips everyone up: your **model** is Gemini, but the **trace dashboard** is OpenAI's. Exporting traces needs an OpenAI key even though inference never touches OpenAI.

```

import os
from dotenv import load_dotenv, find_dotenv
from agents import Agent, Runner, function_tool, set_tracing_export_api_key

load_dotenv(find_dotenv())

# Inference runs on Gemini; this key is only used to upload traces
set_tracing_export_api_key(os.getenv("OPENAI_API_KEY", ""))

@function_tool
def get_weather(city: str) -> str:
    """A simple function to get the weather for a user."""
    return f"The weather for {city} is sunny."

agent = Agent(
    name="WeatherAgent",
    instructions="You are a helpful assistant.",
    model=llm_model,
    tools=[get_weather],
)

result = Runner.run_sync(agent, "What's the weather in Karachi?")
print(result.final_output)

```

Note what is **missing**: no `set_tracing_disabled(...)`. Run it, then open <https://platform.openai.com/traces> and you'll see the run, the tool call, and the model's second pass over the tool result.

Note: If the key is missing the SDK doesn't crash — it prints `OPENAI_API_KEY is not set, skipping trace export` and carries on. An empty dashboard usually means that line scrolled past you.

Try it: group several runs into one trace

By default every `Runner.run(...)` is its own trace. Wrap them to get one story instead of three:

```

import asyncio
from agents import trace

async def main():
    with trace("Joke workflow"):
        first = await Runner.run(agent, "Tell me a joke")
        second = await Runner.run(agent, f"Rate this joke: {first.final_output}")
        print(f"Joke: {first.final_output}")
        print(f"Rating: {second.final_output}")

asyncio.run(main())

```

Both runs now appear as spans under a single **Joke workflow** trace, in order, with timings.

Turning it back off

`set_tracing_disabled(True)` kills tracing for the whole process; `RunConfig(tracing_disabled=True)` kills it for one run. If you want the timeline but not the content, `RunConfig(trace_include_sensitive_data=False)` keeps prompts and outputs out of the upload.

Part 11 — Agents as Tools: One Agent Calling Another

In Part 5 a tool was a Python function. A tool can also be **another agent**. You keep one agent in charge of the conversation and let it call specialists the way it calls `get_weather` — each with its own tight instructions, none of them taking over the conversation.

The core idea

`as_tool()` wraps any agent so another agent can call it:

```

from agents import Agent, Runner

spanish = Agent(
    name="Spanish Translator",
    instructions="Translate what the user says into Spanish. Only output Spanish.",
    model=llm_model,
)

summarizer = Agent(
    name="Summarizer",
    instructions="Summarize the given text in 2 short bullet points.",
    model=llm_model,
)

coach = Agent(
    name="Writing Coach",
    instructions=(
        "You help users improve messages.\n"
        "- If they ask for Spanish, call translate_to_spanish.\n"
        "- If they ask for a summary, call summarize_text.\n"
        "- Otherwise, give a short tip yourself."
    ),
    model=llm_model,
    tools=[
        spanish.as_tool(
            tool_name="translate_to_spanish",
            tool_description="Translate user text to Spanish.",
        ),
        summarizer.as_tool(
            tool_name="summarize_text",
            tool_description="Summarize text in 2 bullets.",
        ),
    ],
)

result = Runner.run_sync(coach, "Please translate to Spanish: I love hands-on examples.")
print(result.final_output)

```

The reply comes from the **coach**, not from the translator. The specialist ran, returned its text, and the coach wrote the final answer around it — exactly the loop from Part 5, with an agent where the Python function used to be.

- The generated tool takes a single string parameter, `input`. The orchestrator writes that string itself, so its `instructions` decide what the specialist actually receives.
- `tool_description` is what the orchestrator reads when choosing — same rule as the docstring in Part 5.
- Each specialist keeps its own `model` and `model_settings`, so a cheap fast model can do translation while the orchestrator runs on a stronger one.

Try it: full control with a function tool

When you need more than the wrapper offers — a turn limit, post-processing, a different run config — write an ordinary `@function_tool` and call `Runner.run` inside it:

```
from agents import function_tool

proofreader = Agent(
    name="Proofreader",
    instructions="Fix grammar and punctuation. Reply only with the corrected text.",
    model=llm_model,
)

@function_tool
async def proofread_text(text: str) -> str:
    """Fix grammar and punctuation; return only the corrected text."""
    result = await Runner.run(proofreader, text, max_turns=3)
    return str(result.final_output)
```

Agent-as-tool is not a handoff

A **handoff** transfers the conversation: the second agent takes the mic and answers the user directly. **Agent-as-tool** keeps the first agent in charge — it borrows an answer, then replies in its own voice. Use tools when you want one consistent personality with borrowed skills; handoffs are for genuinely passing the job on — which is exactly what Part 12 does.

Part 12 — Handoffs: Transferring the Conversation

Part 11 ended with the distinction: a tool borrows an answer, a **handoff** passes the job on. This is the handoff. The first agent stops being the one talking to the user, and a specialist takes over the rest of the conversation.

The core idea

List the agents an agent may transfer to in `handoffs=`. The SDK turns each one into a tool the model can call — a **Study Coach** becomes `transfer_to_study_coach` — so routing is just tool selection, using everything you already know from Part 5.

```

import asyncio
from agents import Agent, Runner, handoff

fitness_coach = Agent(
    name="Fitness Coach",
    instructions=(
        "You're a running coach. Ask 1-2 quick questions, then give a week plan. "
        "Keep it simple and encouraging. No medical advice."
    ),
    model=llm_model,
)

study_coach = Agent(
    name="Study Coach",
    instructions=(
        "You're a study planner. Ask for the current routine, then give a "
        "one-week schedule. Keep steps small and doable."
    ),
    model=llm_model,
)

router = Agent(
    name="Coach Router",
    instructions=(
        "Route the user:\n"
        "- running, workout, stamina -> hand off to the Fitness Coach.\n"
        "- exams, study plan, focus, notes -> hand off to the Study Coach.\n"
        "After the handoff the specialist continues the conversation."
    ),
    model=llm_model,
    handoffs=[study_coach, handoff(fitness_coach)],
)

async def main():
    result = await Runner.run(router, "I want to run 5km in 8 weeks. Can you help?")
    print(result.final_output)
    print("answered by:", result.last_agent.name)

asyncio.run(main())

```

Both forms in that list are legal: a bare agent, or `handoff(agent)` when you want to customise it. The router's own reply never reaches the user — the coach's does.

Proving the handoff happened

`result.last_agent` is the agent that actually answered, and it's how you know routing worked. For the full evidence, look through `result.new_items` for a `HandoffCallItem` (the router asked) followed by a `HandoffOutputItem` (the specialist accepted).

Note: The generated tool is named from the agent's name, lowercased with underscores — `Fitness Coach` becomes `transfer_to_fitness_coach`. Rename an agent and you rename its handoff tool, which quietly invalidates any instructions that referred to the old name.

Try it: keep talking to the specialist

A handoff transfers the conversation, so the *next* turn should go straight to the specialist rather than back through the router. Back inside `main()`, after that first run:

```
follow_up = result.to_input_list() + [
    {"role": "user", "content": "Right now I jog about 2 km, 3 days a week."}
]
second = await Runner.run(result.last_agent, follow_up)
print(second.final_output)
```

`to_input_list()` hands you the whole conversation so far in the format `Runner.run` expects. Pair it with `result.last_agent` and the coach picks up mid-thought instead of reintroducing itself.

Customising a handoff

`handoff(agent, ...)` takes `tool_name_override` and `tool_description_override` when the generated names read badly to the model, `on_handoff` for a callback that fires at the moment of transfer (logging, warming a cache), `input_type` to demand structured data from the router, and `input_filter` to trim what history the specialist inherits. Start with bare agents; reach for these when routing misbehaves — Part 20 covers all of them.

Part 13 — Advanced Tool Control: Stopping, Gating, and Failing Well

Part 5 gave your agent tools. This part is about control: when the loop stops, which tools exist for which user, and what happens when a tool blows up. These are the settings that separate a demo from something you'd let other people run.

Stopping the loop early

By default a tool's output goes back to the LLM, which decides what to do next — that's `tool_use_behavior="run_llm_again"`. Two other modes cut that short. `"stop_on_first_tool"` makes the first tool's raw output the final answer. `StopAtTools` names the tools that end the workflow:

```

from agents import Agent, Runner, function_tool, StopAtTools

@function_tool
def get_weather(city: str) -> str:
    """A simple function to get the weather for a user."""
    return "Sunny"

@function_tool
def get_travel_plan(city: str) -> str:
    """Plan travel for the given city."""
    return "Travel plan is not available"

agent = Agent(
    name="TravelAgent",
    instructions="You are a helpful assistant.",
    model=llm_model,
    tools=[get_weather, get_travel_plan],
    tool_use_behavior=StopAtTools(stop_at_tool_names=["get_travel_plan"]),
)

result = Runner.run_sync(agent, "Make me a travel plan for Lahore")
print(result.final_output)

```

`final_output` is now the tool's own string, word for word. The model never saw it, never softened it, never added a sentence around it. That's the point when a tool already returns exactly what the user should get — and the trap when it doesn't.

The safety net: `max_turns`

`max_turns` caps how many times the loop may call the LLM. Reach it and the SDK raises, rather than burning your quota in a circle:

```

from agents.exceptions import MaxTurnsExceeded

try:
    result = Runner.run_sync(agent, "Research AI agents thoroughly.", max_turns=3)
    print(result.final_output)
except MaxTurnsExceeded:
    print("Hit the turn limit – stopping instead of looping forever.")

```

Count the turns your workflow actually needs before setting this. A tool call plus the model's reading of the result is two turns, so `max_turns=1` stops before the agent can ever use a tool.

Tools that appear and disappear

`is_enabled` decides whether a tool is offered to the model at all. Pass `False` to shelve a tool during maintenance, or a function to decide per run — which is how you build permissions:

```

from dataclasses import dataclass
from agents import RunContextWrapper

@dataclass
class UserContext:
    user_id: str
    subscription_tier: str = "free"

def premium_only(ctx: RunContextWrapper[UserContext], agent: Agent) -> bool:
    return ctx.context.subscription_tier in ("premium", "enterprise")

@function_tool(is_enabled=premium_only)
def deep_research(topic: str) -> str:
    """Run an expensive deep research pass on a topic."""
    return f"Deep research on {topic}: ..."

```

A disabled tool isn't refused — it's never mentioned to the model, so the model can't ask for it and can't apologise for it either. The check reads the same context object you built in Part 7.

Failing where the model can see it

A tool that raises kills the run. A tool that *returns* its error keeps the conversation alive and gives the model something to work with:

```

@function_tool
def divide(a: int, b: int) -> str:
    """Divide two numbers."""
    try:
        return str(a / b)
    except ZeroDivisionError:
        return "Error: cannot divide by zero. Ask the user for a different number."

```

The model reads that sentence and asks a better question. For heavier needs — logging, retries, a standard error shape across many tools — `@function_tool(failure_error_function=...)` intercepts exceptions centrally instead.

Part 14 — Structured Output: Answers Your Code Can Use

Every run so far ended in `result.final_output` as prose. Prose is fine for a human and useless to an `if` statement. `output_type` makes the agent return a **typed object** instead — the same call, but the answer arrives parsed.

The core idea

Describe the shape you want as a Pydantic model and hand it to the Agent:

```

from pydantic import BaseModel
from agents import Agent, Runner

class PersonInfo(BaseModel):
    name: str
    age: int
    occupation: str

agent = Agent(
    name="InfoCollector",
    instructions="Extract person information from the user's message.",
    model=llm_model,
    output_type=PersonInfo,
)

message = "Hi, I'm Alice, I'm 25 years old and I work as a teacher."
result = Runner.run_sync(agent, message)

print(type(result.final_output))      # <class 'PersonInfo'>
print(result.final_output.name)       # Alice
print(result.final_output.age + 1)    # 26 – a real int, not the text "25"

```

No parsing, no regex, no "please reply in JSON" in the instructions. `final_output` is a `PersonInfo` instance and your editor knows its fields.

What the SDK is doing for you

Your model becomes a JSON schema that goes to the provider alongside the prompt, and the reply is validated against it before you ever see it. Schemas are sent in **strict** mode, which lists *every* field as required — including ones with a Python default. A default such as `rainfall: float | None = None` means your code still parses when the field is missing; it does not invite the model to omit it. Lists and nested models work the same way, and `output_type=list[str]` is legal too.

When the model returns junk

Validation is the point, so failure is loud. A reply that doesn't fit the schema raises `ModelBehaviorError: Invalid JSON when parsing model output` rather than handing you a half-parsed object:

```

from agents.exceptions import ModelBehaviorError

try:
    result = Runner.run_sync(agent, "Tell me a story about a dragon.")
    print(result.final_output.name)
except ModelBehaviorError as e:
    print("The model didn't produce the shape we asked for:", e)

```

If you see this often, the fix is usually a lower `temperature` from Part 6 and field names the model can actually infer from the prompt — not a longer instruction.

Note: Schemas are sent in strict mode, and not every provider accepts every schema. If a valid model is rejected, wrap it as `AgentOutputSchema(MyModel, strict_json_schema=False)` and pass that as the `output_type` instead.

Why this matters for the rest of the guide

A typed answer is what lets one agent's output drive Python instead of another prompt: routing on a boolean, storing a score, branching on a category. It's also what makes the guardrail in Part 15 possible — a checker agent that answers `weather_related: bool` gives you something to test, where prose would only give you something to read.

Part 15 — Guardrails: Refusing Bad Input and Bad Output

Every part so far made your agent more capable. This one makes it say no. A **guardrail** is a check that runs alongside the agent and can abort the whole run — before an expensive model ever sees a junk request, or before a bad answer ever reaches your user.

Two checkpoints

Input guardrails run before the agent does, on the first agent in the run. **Output guardrails** run after, on the last agent. Each returns a `GuardrailFunctionOutput`; when its `tripwire_triggered` is `True`, the SDK raises and the run stops there.

The cheapest useful guardrail isn't an LLM at all:

```

from agents import (Agent, Runner, GuardrailFunctionOutput, RunContextWrapper,
                    input_guardrail, InputGuardrailTripwireTriggered)

@input_guardrail
async def homework_check(
    ctx: RunContextWrapper, agent: Agent, user_input
) -> GuardrailFunctionOutput:
    text = user_input if isinstance(user_input, str) else str(user_input)
    return GuardrailFunctionOutput(
        output_info={"checked": True},
        tripwire_triggered="homework" in text.lower(),
    )

tutor = Agent(
    name="Tutor",
    instructions="Explain concepts. Never complete assignments.",
    model=llm_model,
    input_guardrails=[homework_check],
)

try:
    result = Runner.run_sync(tutor, "do my homework for me")
    print(result.final_output)
except InputGuardrailTripwireTriggered as e:
    print("Blocked before the model ran:", e.guardrail_result.output.output_info)

```

Run that and the exception arrives with **no network call at all** — you were never billed for the request. That's the argument for guardrails in one line.

Guarding with a second agent

For judgements a keyword can't make, run a small, cheap agent as the guardrail and have it answer in a fixed shape:

```

from pydantic import BaseModel

class WeatherCheck(BaseModel):
    weather_related: bool
    reason: str | None = None

weather_sanitizer = Agent(
    name="WeatherSanitizer",
    instructions="Decide whether the user's message is a weather question.",
    model=llm_model,
    output_type=WeatherCheck,
)

@input_guardrail
async def weather_only(
    ctx: RunContextWrapper, agent: Agent, user_input
) -> GuardrailFunctionOutput:
    res = await Runner.run(weather_sanitizer, user_input)
    return GuardrailFunctionOutput(
        output_info=res.final_output.reason,
        tripwire_triggered=res.final_output.weather_related is False,
    )

```

That's Part 14's `output_type` doing the work: the sanitizer hands back a parsed `WeatherCheck`, so `res.final_output.weather_related` is a real boolean you can branch on rather than a sentence you'd have to interpret. Guard with a small model — a guardrail that costs more than the agent it protects is not a guardrail.

Output guardrails

`@output_guardrail` takes `(ctx, agent, output)` and goes on `output_guardrails=[...]`. Catch `OutputGuardrailTripwireTriggered` for it. Use it for the checks that only make sense on a finished answer: leaked keys, an off-brand tone, a refusal you promised never to send.

Note: A tripwire raises. If you don't wrap the run in `try/except`, a blocked request crashes your program rather than politely declining — which is exactly what a beginner's first guardrail does.

Part 16 — Lifecycle Hooks: Watching the Loop from Inside

Part 10 gave you the trace after the fact. Hooks give you the run *as it happens*, in your own Python: a callback at every stage of the loop, where you can log, count, time, or update a UI.

Subclass, override, attach

`AgentHooks` has one method per stage. Override the ones you care about and hand an instance to the agent:

```

from agents import Agent, Runner, AgentHooks, function_tool

class LoudHooks(AgentHooks):
    def __init__(self, label: str):
        self.label = label

    async def on_start(self, context, agent):
        print(f"[{self.label}] {agent.name} is now responsible for the answer")

    async def on_tool_start(self, context, agent, tool):
        print(f"[{self.label}] calling {tool.name}")

    async def on_tool_end(self, context, agent, tool, result):
        print(f"[{self.label}] {tool.name} returned {result}")

    async def on_end(self, context, agent, output):
        print(f"[{self.label}] finished")

@function_tool
def get_weather(city: str) -> str:
    """A simple function to get the weather for a user."""
    return f"The weather for {city} is sunny."

agent = Agent(
    name="WeatherAgent",
    instructions="You are a helpful assistant.",
    model=llm_model,
    tools=[get_weather],
    hooks=LoudHooks("weather"),
)

result = Runner.run_sync(agent, "What's the weather in Karachi?")
print(result.final_output)

```

The full set is `on_start`, `on_llm_start`, `on_llm_end`, `on_tool_start`, `on_tool_end`, `on_handoff` and `on_end`. For that weather question they fire in that order, and you can finally see the two-pass loop from Part 5: the model is called, the tool runs, then the model is called again with the result.

`on_start` and `on_llm_start` are easy to confuse. `on_start` fires **once**, when this agent becomes the one responsible for answering. `on_llm_start` fires **every time** that agent calls the model — twice in the run above, because a tool came back with a result the model had to read. Count tokens in `on_llm_start`; set up per-agent state in `on_start`.

Agent hooks or run hooks

`AgentHooks` belongs to one agent — attach different hooks to different agents and each reports on itself. `RunHooks` covers the whole run, handed to `Runner.run(..., hooks=...)`, and fires across every agent in it, including after a Part 12 handoff. That's Part 17.

Note: The two classes don't share method names. `AgentHooks` has `on_start` and `on_end`; `RunHooks` has `on_agent_start` and `on_agent_end`. Override the wrong pair and your callback is silently never called.

What they're actually for

Counting tokens and cost per agent, timing which tool is slow, pushing "thinking..." updates to a UI, writing an audit log of every tool call. Keep them fast: hooks run inside the loop, so anything slow you do in `on_tool_end` is time the user spends waiting.

Part 17 — Run Lifecycle Hooks: Watching Every Agent at Once

`AgentHooks` from Part 16 belongs to one agent. The moment a handoff moves the conversation (Part 12), those callbacks go quiet — the specialist has its own hooks, or none at all. `RunHooks` sits one level up and watches the entire run, every agent in it, from the first prompt to the last answer.

One monitor for the whole run

Subclass `RunHooks`, then hand the instance to the runner instead of to an agent:

```

from agents import Agent, Runner, RunHooks, function_tool

class SystemMonitor(RunHooks):
    def __init__(self):
        self.timeline = []

    async def on_agent_start(self, context, agent):
        self.timeline.append(f"start {agent.name}")

    async def on_tool_start(self, context, agent, tool):
        self.timeline.append(f"{agent.name} calls {tool.name}")

    async def on_handoff(self, context, from_agent, to_agent):
        self.timeline.append(f"handoff {from_agent.name} -> {to_agent.name}")

    async def on_agent_end(self, context, agent, output):
        self.timeline.append(f"end {agent.name}")

@function_tool
def get_weather(city: str) -> str:
    """A simple function to get the weather for a user."""
    return f"The weather for {city} is sunny."

news_agent = Agent(
    name="NewsAgent",
    instructions="Answer news questions.",
    model=llm_model,
)

weather_agent = Agent(
    name="WeatherAgent",
    instructions="Talk about weather. Let the NewsAgent handle news questions.",
    model=llm_model,
    tools=[get_weather],
    handoffs=[news_agent],
)

monitor = SystemMonitor()
result = Runner.run_sync(weather_agent, "What's the latest news on AI coding tools?",
                        hooks=monitor)
print(result.final_output)
print(monitor.timeline)

```

Ask a news question and `monitor.timeline` reads roughly `start WeatherAgent, handoff WeatherAgent -> NewsAgent, start NewsAgent, end NewsAgent` — one ordered story spanning both agents. Attach `AgentHooks` to `WeatherAgent` instead and the trail stops at the handoff.

The two classes don't line up

They cover the same events, but they are not interchangeable:

- **AgentHooks** uses `on_start` and `on_end`; **RunHooks** uses `on_agent_start` and `on_agent_end`, and fires them once per agent that takes part.
- `on_handoff` takes different arguments. **AgentHooks** receives `(context, agent, source)` — *I was just handed this, and here's who sent it.* **RunHooks** receives `(context, from_agent, to_agent)` — *this one left, that one arrived.*
- `on_tool_start`, `on_tool_end`, `on_llm_start` and `on_llm_end` have the same shape on both.

Note: The runner keyword is `hooks=`, the same name used for the per-agent hooks on `Agent(...)`. Which class you passed is what decides which methods get called — there is no separate `run_hooks=` argument.

Which one to reach for

Use **AgentHooks** when you care about one agent's own behaviour — a specialist you're tuning, a tool you suspect is slow. Use **RunHooks** when the question is about the request as a whole: total tokens and cost across every agent, an audit log that survives handoffs, a progress indicator that keeps updating after the conversation changes hands. They compose — a run can carry a monitor while individual agents carry their own hooks, and both fire.

Part 18 — Custom Runners: Wrapping Every Run You Make

Runner has been a black box since Part 3. It doesn't have to be. **AgentRunner** is the class behind it, and you can subclass it to put your own code on both sides of every run in the process — without editing a single agent definition.

Subclass it, then register it

```
import asyncio
from agents import Agent, Runner, OpenAIChatCompletionsModel, AsyncOpenAI
from agents.run import AgentRunner, set_default_agent_runner

class CustomAgentRunner(AgentRunner):
    async def run(self, starting_agent, input, **kwargs):
        # before: routing, auth, input rewriting, load balancing
        print(f"[infra] running {starting_agent.name}")

        result = await super().run(starting_agent, input, **kwargs)

        # after: analytics, cost accounting, persistence
        print(f"[infra] finished {starting_agent.name}")
        return result

set_default_agent_runner(CustomAgentRunner())

agent = Agent(
    name="Assistant",
    instructions="You only respond in haikus.",
    model=llm_model,
)

async def main():
    result = await Runner.run(agent, "Tell me about recursion in programming.")
    print(result.final_output)

asyncio.run(main())
```

`Runner.run(...)` looks exactly the same at the call site. Underneath, it now goes through your class, and so does every other run anywhere in the program — including runs started by an `as_tool` specialist from Part 11 or by a handoff from Part 12.

Where this sits next to the tools you already have

Three different layers, and it's worth keeping them straight:

- **Guardrails** (Part 15) decide whether a run is allowed to happen at all, and abort it.
- **Hooks** (Parts 16–17) observe a run that is already happening, without changing it.
- **A custom runner** *wraps* the run. It sees the input before the SDK does, the result after, and it can change either — or replace the execution entirely.

What it's actually for

This is infrastructure, not application code: one place to attach request IDs, meter usage per customer, enforce a tenant's rate limit, persist every run to a database, or route work to a different pool of workers. If you find yourself pasting the same three lines around every `Runner.run` call in a codebase, that's the signal to move them in here.

Note: `set_default_agent_runner` is process-wide global state. It's the right shape for a server's startup code, and the wrong shape for a library — set it once at boot, not per request.

Part 19 — Chainlit: Putting a Chat UI on Your Agent

Every agent so far talked to a terminal. **Chainlit** is a Python framework that gives it a real chat interface — message bubbles, streaming, file uploads, per-user sessions — without you writing any frontend code. Two decorators and your Part 5 agent is a web app.

Setup

Chainlit is a new dependency, and it ships its own run command:

```
uv add chainlit
uv run chainlit run chatbot.py -w
```

That serves the app on `http://localhost:8000`; `-w` reloads on save. The smallest possible app is an echo bot, and it's worth running first to see the loop:

```
import chainlit as cl

@cl.on_message
async def main(message: cl.Message):
    await cl.Message(content=f"Received: {message.content}").send()
```

Wiring your agent in

`@cl.on_chat_start` fires once when someone opens the page — build the agent there and stash it in `cl.user_session`, which is private to that browser session. `@cl.on_message` fires per message:

```

import os
from typing import cast
import chainlit as cl
from dotenv import load_dotenv, find_dotenv
from agents import Agent, Runner, AsyncOpenAI, OpenAIChatCompletionsModel

load_dotenv(find_dotenv())

@cl.on_chat_start
async def start():
    external_client = AsyncOpenAI(
        api_key=os.getenv("GEMINI_API_KEY"),
        base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
    )
    llm_model = OpenAIChatCompletionsModel(
        model="gemini-2.5-flash", openai_client=external_client,
    )
    agent = Agent(name="Assistant", instructions="You are a helpful assistant.",
        model=llm_model)

    cl.user_session.set("agent", agent)
    cl.user_session.set("history", [])
    await cl.Message(content="Ask me anything.").send()

@cl.on_message
async def handle(message: cl.Message):
    agent = cast(Agent, cl.user_session.get("agent"))
    history = cl.user_session.get("history") or []
    history.append({"role": "user", "content": message.content})

    thinking = cl.Message(content="Thinking...")
    await thinking.send()

    result = await Runner.run(agent, history)

    thinking.content = result.final_output
    await thinking.update()
    cl.user_session.set("history", result.to_input_list())

```

Two things make this a real chatbot rather than a demo. The agent is created once per session, not once per message. And `result.to_input_list()` — the same call you used for handoffs in Part 12 — feeds the whole conversation back in next turn, which is the only reason the bot remembers anything.

Note: Use `await Runner.run(...)` here, never `Runner.run_sync(...)`. Chainlit handlers are already inside an event loop, and `run_sync` starts its own — it raises `RuntimeError: AgentRunner.run_sync() cannot be called when an event loop is already running`. This is Part 3's distinction finally biting.

Where to take it

`cl.user_session` is per-browser-session state, which pairs naturally with the local context from Part 7: build a `UserInfo` at chat start, store it, and pass it as `context=` on every run. From there the same app absorbs everything else in this guide — tools, handoffs, guardrails — with no change to the Chainlit layer at all.

Part 20 — Advanced Handoffs: Controlling the Transfer

Part 12 put bare agents in `handoffs=`. `handoff(agent, ...)` is the object underneath, and it turns routing into something you control: rename the generated tool, fire a callback on transfer, demand structured input from the router, trim the history the specialist inherits, and switch a route off.

A callback, and a typed reason for the transfer

`on_handoff` runs at the moment of transfer — the place for logging, a metric, or warming a cache. Pair it with `input_type` and the router must fill in a structure before it is allowed to route:

```
from pydantic import BaseModel
from agents import Agent, Runner, RunContextWrapper, handoff

class NewsRequest(BaseModel):
    topic: str
    reason: str

def on_news_transfer(ctx: RunContextWrapper, data: NewsRequest) -> None:
    print(f"[handoff] news about {data.topic} because {data.reason}")

news_agent = Agent(
    name="NewsAgent",
    instructions="Answer tech news questions.",
    model=llm_model,
)

weather_agent = Agent(
    name="WeatherAgent",
    instructions="Weather expert. Send tech and news questions to the news desk.",
    model=llm_model,
    handoffs=[
        handoff(
            news_agent,
            tool_name_override="ask_the_news_desk",
            on_handoff=on_news_transfer,
            input_type=NewsRequest,
        )
    ],
)
```

The router now has to say *what* it is escalating and *why*, in fields you can read in Python. `tool_name_override` matters more than it looks: the generated name is what the model sees, so a clearer name routes better.

Note: The callback's signature must match. With an `input_type`, `on_handoff` takes two arguments; without one, it takes exactly one. Get it wrong and the SDK raises at construction time — `UserError: on_handoff must take two arguments: context and input`, or the one-argument version of the same message.

Trimming what the specialist inherits

By default the specialist receives the conversation so far, tool calls included. An `input_filter` rewrites that first. It is handed a `HandoffInputData` carrying `input_history`, `pre_handoff_items` and `new_items`, and returns the version the specialist will actually see:

```
from agents.handoffs import HandoffInputData
from agents.extensions import handoff_filters

def summarised_transfer(data: HandoffInputData) -> HandoffInputData:
    return HandoffInputData(
        input_history="Get the latest tech news.",
        pre_handoff_items=(),
        new_items=(),
    )

# your own filter, or the one the SDK already ships:
handoff(news_agent, input_filter=summarised_transfer)
handoff(news_agent, input_filter=handoff_filters.remove_all_tools)
```

Replacing the history with one sentence is brutal — the specialist starts nearly fresh. `remove_all_tools` is the gentler default: the conversation survives, the tool noise does not. Filter too hard and the specialist asks something the user already answered.

Switching a route off: `is_enabled`

A handoff can be hidden the same way a tool can. Pass `False` to shelve it, or a function to decide per run from the context:

```
from agents import function_tool

# 1. a route the model cannot take unless the context allows it
handoff(expert_agent, is_enabled=lambda ctx, agent: ctx.context.has_permission)

# 2. the same idea on a plain tool, from Part 13
@function_tool(is_enabled=False)
def seasonal_offer() -> str:
    """Switched off out of season."""
    return "not available"

# 3. and on an agent exposed as a tool, from Part 11
expert_agent.as_tool(
    tool_name="ask_expert",
    tool_description="Ask the recursion expert.",
    is_enabled=False,
)
```

One flag, three places: a tool, a handoff, and an agent-as-tool. In every case a disabled thing is not refused — it is never mentioned to the model, so it cannot be asked for and cannot be apologised for. A sync function is fine; the SDK wraps it, so the stored `is_enabled` is awaited either way.

When to reach for any of this

Start with bare agents. Add `tool_name_override` when the model routes to the wrong specialist, `input_type` when you need to log *why*, `input_filter` when transfers get expensive, and `is_enabled` when a route depends on who is asking. Reaching for all of them at once is how routing stops being explainable.

Recap

Concept	What it gives you
<code>Agent(name, instructions, model, tools)</code>	Defines <i>who</i> the agent is and what it can use
<code>Runner.run_sync / run / run_streamed</code>	Actually executes the agent against an LLM
<code>asyncio</code>	Lets LLM calls (network I/O) run without blocking your whole program
Agent / Run / Global model config	Three scopes for choosing <i>which</i> LLM backs an agent
<code>@function_tool</code>	Turns a Python function into something the agent can call mid-conversation
<code>ModelSettings(temperature, tool_choice, max_tokens)</code>	Tunes <i>how</i> the chosen model behaves on a given run
<code>RunContextWrapper + context=</code>	Passes your own data to tools without ever showing it to the LLM
<code>instructions=<callable></code>	Rebuilds the system prompt on every turn, from context or state
<code>agent.clone(...)</code>	Produces a variant of an agent, overriding only the fields you name
<code>trace(...)</code> / traces dashboard	Records every step of a run so you can inspect it after the fact
<code>agent.as_tool(...)</code>	Turns a specialist agent into a tool another agent can call
<code>handoffs=[...] + result.last_agent</code>	Transfers the conversation to a specialist and tells you who answered
<code>tool_use_behavior, max_turns, is_enabled</code>	Control over when the loop stops and which tools exist
<code>output_type=<model></code>	Returns a validated Pydantic object instead of prose
<code>@input_guardrail / @output_guardrail</code>	Aborts a run before a bad request is paid for, or a bad answer is sent
<code>AgentHooks</code>	Callbacks at every stage of one agent's work
<code>RunHooks + Runner.run(..., hooks=...)</code>	The same events for every agent in a run, handoffs included
<code>AgentRunner + set_default_agent_runner</code>	Wraps every run in the process with your own before/after code

Concept	What it gives you
<code>@cl.on_chat_start / @cl.on_message</code>	Puts a real chat UI in front of the agent, one session per user
<code>handoff(agent, ...)</code>	Renames, gates, logs and filters a transfer instead of accepting the defaults
<code>is_enabled</code>	One flag that hides a tool, a handoff, or an agent-as-tool from the model

You now have everything needed to build the **First Coded Agent** project: an agent, backed by a provider of your choice, that can reason about a request and reach for a tool when it needs one.

Part 21 — Practice This with Claude Code

Reading code is not the same as building it. This course's approach is to **learn by prompting Claude Code** — Anthropic's agentic coding CLI — and reviewing what it produces, rather than typing everything from scratch or copy-pasting blindly. Open a terminal in your project folder, run `claude`, and work through the prompts below **in order**. Each one builds on the file Claude Code just created or edited for you.

Don't just accept the output — read the diff, ask "why did you do it that way?", and only move to the next prompt once you understand the code you now have.

Prompt-by-part

Part	Prompt to give Claude Code
0. Setup	Scaffold a uv-managed Python project called hello-agent with openai-agents and python-dotenv as dependencies. Create a .env file with empty OPENAI_API_KEY and GEMINI_API_KEY entries, and make sure .env is gitignored.
1. OpenAI key	Create main.py using the OpenAI Agents SDK: an Agent named Assistant with instructions "You are a helpful assistant", run synchronously against the prompt "Write a haiku about recursion in programming." Load OPENAI_API_KEY from .env. Then explain what Agent and Runner.run_sync each do.
2. Gemini key	Modify main.py so the same Agent uses Gemini's gemini-2.5-flash model instead of OpenAI's default – wire up AsyncOpenAI with Gemini's OpenAI-compatible base_url and wrap it in OpenAIChatCompletionsModel. Explain line-by-line why swapping the provider didn't require touching Agent or Runner.
3. Runner & asyncio	Rewrite main.py to call Runner.run inside an async def main() instead of run_sync, executed via asyncio.run(). Then add a second version that fires two different prompts at the same agent concurrently with asyncio.gather and prints both results. Explain why the concurrent version is faster.
4. Model configuration	Refactor my agent so the Gemini model is set at the Run level via RunConfig instead of the Agent level. Show me the same agent being called once with the Gemini run_config and once without one, so I can see the difference. Explain when I'd pick Run-level over Agent-level.
5. Tool use	Add a @function_tool called get_weather(city: str) -> str returning a hardcoded string, and a second tool add(a: int, b: int) -> int. Wire both into the agent's tools list and run it against "What's 42 plus 58, and what's the weather in Lahore?" Then walk me through, step by step, what happened between my prompt and the final answer, including which tools got called and why.
6. Model settings	Take my weather-and-add agent and run the same question three times with model_settings=ModelSettings(tool_choice="auto"), then "required", then "none". Print all three answers side by side and explain what changed and why the "none" answer is different.
7. Local context	Add a UserInfo dataclass with name and uid, pass it to Runner.run via context=, and add a tool that reads wrapper.context.name. Then print the tool's params_json_schema and explain why the wrapper parameter isn't in it.
8. Dynamic instructions	Replace my agent's instructions string with a function taking (context, agent) that greets the user by name from context and mentions the agent's own name. Print the resolved system prompt before the model call, then convert the function into a callable class that counts interactions.

Part	Prompt to give Claude Code
9. Agent cloning	Create a base agent with temperature 0.7 and one tool, then clone it into a creative variant and a precise variant. Prove to me with is-comparisons which attributes are shared between base and clone and which are not.
10. Tracing	Remove set_tracing_disabled from my project, call set_tracing_export_api_key with my OpenAI key, and wrap two consecutive Runner.run calls in a single with trace("...") block. Then tell me exactly which spans I should expect to see on platform.openai.com/traces and in what order.
11. Agents as tools	Build a Writing Coach agent that owns two specialists wrapped with as_tool: a Spanish translator and a summarizer. Run all three of my requests through it, then show me which tool fired for each and explain why the final answer still comes from the coach.
12. Handoffs	Convert my Writing Coach's two as_tool specialists into real handoffs instead. Run the same three requests, print result.last_agent for each, and show me the HandoffCallItem and HandoffOutputItem in result.new_items. Then tell me which of the two designs I should keep and why.
13. Advanced tool control	Add StopAtTools to my agent so the loop ends on a finalizing tool, wrap a run in try/except MaxTurnsExceeded with max_turns=2, and gate one tool behind is_enabled reading a subscription tier from my context. Run it once as a free user and once as premium and show me the difference in the tools the model was offered.
14. Structured output	Give my agent an output_type Pydantic model with three typed fields, run it, and print type(result.final_output) plus one field used in arithmetic to prove it isn't a string. Then feed it a prompt it can't answer in that shape and show me the exception.
15. Guardrails	Add an input guardrail that trips on off-topic requests without calling any model, and catch InputGuardrailTripwireTriggered so my program declines politely instead of crashing. Then prove to me from the trace that no model call was billed for the blocked request.
16. Lifecycle hooks	Attach an AgentHooks subclass that prints on_start, on_tool_start, on_tool_end and on_end, run one question that needs a tool, and show me the order the hooks fired in. Then explain what that order tells me about the agent loop.
17. Run lifecycle hooks	Give my two-agent handoff setup a RunHooks subclass that records on_agent_start, on_handoff and on_agent_end into one list, pass it as hooks= on the run, and print the list. Then attach AgentHooks to the first agent instead and show me where that trail stops.
18. Custom runners	Subclass AgentRunner so it prints the agent name and elapsed time around every run, register it with set_default_agent_runner, then run an agent that hands

Part	Prompt to give Claude Code
	off to another. Show me that the wrapper fired for both runs without me editing either agent.
19. Chainlit	Wrap my Gemini agent in a Chainlit app: build the agent in <code>@cl.on_chat_start</code> , store it in <code>cl.user_session</code> , and answer messages in <code>@cl.on_message</code> using <code>await Runner.run</code> with the history from <code>to_input_list()</code> . Then show me what breaks if I use <code>Runner.run_sync</code> instead.
20. Advanced handoffs	Take my two-agent handoff and give it a <code>tool_name_override</code> , an <code>input_type</code> carrying the reason, an <code>on_handoff</code> callback that logs it, and an <code>input_filter</code> that strips tool calls. Then gate the route behind <code>is_enabled</code> and show me the model's tool list with the gate open and closed.

Debug-it prompts (learn by breaking things)

Understanding deepens when you have to *fix* something, not just read it. Try these after finishing Part 5:

- Deliberately break my Gemini setup by using the wrong `base_url`, show me the exact error it raises, then walk me through how you diagnosed and fixed it.
- Rename the `get_weather` function to `fetch_weather` but leave the docstring and tools list referencing the old name. Run it, show me what breaks, then fix it and explain why the SDK cares about the function name at all.
- Remove the type hint from the `city` parameter in `get_weather`. Run the agent again – does it still work? Explain what the SDK does differently when a tool parameter has no type hint.
- Change my dynamic instructions function so it takes only the context parameter, run it, and show me the exact `TypeError` the SDK raises before fixing it back.
- Give my two `as_tool` specialists the same `tool_name`, run the orchestrator, and show me what happens. Then fix it and explain how the model tells the two tools apart.
- Rename my Fitness Coach agent to Running Coach but leave the router's instructions mentioning `transfer_to_fitness_coach`. Run it, show me how the routing degrades, and explain where that tool name comes from.
- Override `on_agent_start` on an `AgentHooks` subclass instead of `on_start`, attach it, and run the agent. Show me that nothing prints, then explain which class that method actually belongs to.
- Give my handoff an `input_type` but leave `on_handoff` taking a single argument. Show me the `UserError` the SDK raises at construction time, then fix it both ways round.
- Take my local-context example and remove `context=user_info` from the `Runner.run` call, leaving the tool untouched. Run it, show me the exact error, and explain which line of the tool blew up and why.

Capstone prompt

Once you've been through all twenty-one parts, try building something new end-to-end in one shot, then critique what comes back:

```
Using the OpenAI Agents SDK, build a small CLI agent powered by Gemini
(Agent-level configuration) with two tools of your own choosing (not
get_weather or add). Expose it through an async main() run via
asyncio.run(). Then give me three test prompts that demonstrate the
agent choosing between the two tools correctly, and explain the tool-
selection reasoning for each one.
```

Compare what Claude Code builds against Parts 1–20 of this guide — if you can explain every line without asking "why," you've learned the material.

Bonus — `run`, `run_sync` and `run_streamed` in Depth

Part 3 introduced the three ways to start an agent and told you which to reach for. This section opens all three up: what each one does to your program, what happens inside `run_sync`, why one of them explodes inside a web framework, and what the errors mean when you get it wrong.

First, three words that get mixed up

Synchronous code runs one statement at a time; the next line waits for the previous one to finish. **Asynchronous** code can *pause* a function at a marked point and let the thread do something else until the thing it was waiting for is ready. **Non-blocking I/O** is why that matters: when your agent calls an LLM, your program spends almost all its time waiting on the network, not computing. A synchronous call sits there doing nothing. An asynchronous one hands the thread back so other work — another agent, another web request — can use it.

Three terms worth having straight before the rest of this section:

- A **coroutine** is what `async def` creates. Calling it does not run it; it returns an object that must be awaited or scheduled.
- The **event loop** is the scheduler that runs coroutines, pausing each at its `await` points.
- **Blocking** means the calling thread cannot do anything else until the call returns.

`Runner.run()` — the asynchronous one

`Runner.run` is an `async def`. Calling it produces a coroutine; `await` is what actually runs it and hands the thread back to the loop while the LLM responds.

```

import asyncio
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="Answer in one short sentence.",
              model=llm_model)

async def main():
    result = await Runner.run(agent, "What is an AI agent?")
    print(result.final_output)

asyncio.run(main())

```

`asyncio.run(main())` creates an event loop, runs `main()` to completion and closes the loop again. Use this form in anything that already has a loop — a web server, a bot, a UI — and in scripts where you want to run several agents at once.

`Runner.run_sync()` — the synchronous wrapper

`Runner.run_sync` is a plain function. No `async`, no `await`, and your next line does not run until the agent is finished.

```

from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="Answer in one short sentence.",
              model=llm_model)

result = Runner.run_sync(agent, "What is an AI agent?")
print(result.final_output)          # this line waits for the whole agent run

```

The agent loop underneath is still asynchronous — `run_sync` does not reimplement it. It drives the same `Runner.run` coroutine on an event loop and blocks the calling thread until the result arrives.

How `run_sync` works internally

Conceptually, and **not** the SDK's source — the real implementation handles cancellation, sessions and loop reuse that this sketch ignores:

```
# conceptual only
def run_sync(agent, input, **kwargs):
    try:
        asyncio.get_running_loop()
    except RuntimeError:
        pass # good: this thread has no loop running
    else:
        raise RuntimeError("cannot be called when an event loop is already running")

    loop = asyncio.get_event_loop_policy().get_event_loop()
    task = loop.create_task(Runner.run(agent, input, **kwargs))
    return loop.run_until_complete(task) # the calling thread blocks here
```

Step by step, what actually happens:

1. It checks whether an event loop is **already running on this thread**. If one is, it refuses.
2. It takes the thread's **default** event loop, creating one only if the thread has none. Note that it does *not* call `asyncio.run()` — that would create and destroy a fresh loop each time, and helpers that cached loop-bound objects between calls would break.
3. It schedules `Runner.run(...)` on that loop as a task.
4. It calls `run_until_complete`, which blocks the calling thread until the task finishes.
5. It returns the `RunResult`. If the caller interrupts, the scheduled task is cancelled rather than left running on the shared loop.

```
run_sync(agent, "hi")
  |
  v
loop already running on this thread?
  |                               |
  yes                             no
  |                               |
  v                               v
RuntimeError                    take the thread's default loop
                               |
                               v
                             loop.create_task(Runner.run(agent, "hi"))
                               |
                               v
                             loop.run_until_complete(task) <-- calling thread blocks
                               |
                               v
                             RunResult returned
```

`Runner.run_streamed()` — events as they happen

The other two hand you a finished answer. `run_streamed` hands you a **live object** you iterate while the agent is still working, which is how you show text appearing a token at a time.

It is **not** a coroutine. It returns immediately, and the run proceeds in the background — so you do not **await** the call itself, you **async for** over its events.

```
import asyncio
from openai.types.responses import ResponseTextDeltaEvent
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="Explain clearly.", model=llm_model)

async def main():
    result = Runner.run_streamed(agent, "Explain recursion to a beginner.")

    async for event in result.stream_events():
        if event.type == "raw_response_event":
            if isinstance(event.data, ResponseTextDeltaEvent):
                print(event.data.delta, end="", flush=True)
        elif event.type == "run_item_stream_event":
            if event.name == "tool_called":
                print("\n[calling a tool]")

    print("\n\nfinal:", result.final_output)

asyncio.run(main())
```

Three kinds of event arrive, and you filter on **event.type**:

event.type	Class	What it carries
raw_response_event	RawResponsesStreamEvent	data — token deltas straight from the model
run_item_stream_event	RunItemStreamEvent	name and item — a message, tool call or tool output
agent_updated_stream_event	AgentUpdatedStreamEvent	new_agent — a handoff changed who is answering

final_output is only complete once the stream has finished. The **name** values on a run-item event include **message_output_created**, **tool_called**, **tool_output**, **handoff_requested** and **handoff_occured** — that last one is spelled with one **r** in the SDK, which will cost you ten minutes the first time.

Using them with FastAPI

A FastAPI **async def** endpoint is already running inside an event loop. That decides everything.

```

from fastapi import FastAPI
from agents import Agent, Runner

app = FastAPI()
agent = Agent(name="Assistant", instructions="Be brief.", model=llm_model)

@app.post("/ask")                                # correct
async def ask(question: str):
    result = await Runner.run(agent, question)
    return {"answer": result.final_output}

@app.post("/ask-broken")                        # wrong
async def ask_broken(question: str):
    result = Runner.run_sync(agent, question)    # raises at request time
    return {"answer": result.final_output}

```

The broken one fails with:

```

RuntimeError: AgentRunner.run_sync() cannot be called when an event loop is
already running.

```

That is not a bug to work around. `run_sync` wants to drive a loop, and the server is already driving one; blocking it would freeze every other request on that worker. In async web code, always `await`.

The three, side by side

	<code>run()</code>	<code>run_sync()</code>	<code>run_streamed()</code>
Execution model	coroutine	plain function	returns a live stream object
<code>await</code> required	yes, on the call	no	no on the call; <code>async</code> for its events
Event loop	runs on the caller's loop	needs a loop that is not running	runs on the caller's loop
Blocks the thread	no	yes, until finished	no
Streaming	no	no	yes
Reach for it	servers, UIs, concurrency	scripts, notebooks, quick tests	anything a human watches

Common mistakes

Error you see	What you did	Fix
<code>RuntimeError:</code> <code>AgentRunner.run_sync()</code> cannot be called when an event loop is already running.	Called <code>run_sync</code> inside <code>async def</code> — FastAPI, Chainlit, a notebook cell	<code>await Runner.run(...)</code>
<code>TypeError:</code> object <code>RunResultStreaming</code> can't be used in <code>'await'</code> expression	Wrote <code>await</code> <code>Runner.run_streamed(...)</code>	Drop the <code>await</code> ; iterate <code>stream_events()</code>
<code>AttributeError:</code> <code>'coroutine'</code> object has no attribute <code>'final_output'</code>	Forgot <code>await</code> on <code>Runner.run</code>	Add <code>await</code> , and check for a "coroutine was never awaited" warning

Running several agents at once

The real payoff of the async form: two runs in flight together take about as long as the slower one.

```
import asyncio
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="Answer in one line.", model=llm_model)

async def main():
    first, second = await asyncio.gather(
        Runner.run(agent, "What is Python?"),
        Runner.run(agent, "What is an AI agent?"),
    )
    print(first.final_output)
    print(second.final_output)

asyncio.run(main())
```

`run_sync` cannot do this. Each call blocks until it finishes, so two of them cost the sum of both — which is the whole argument for learning the async form properly.